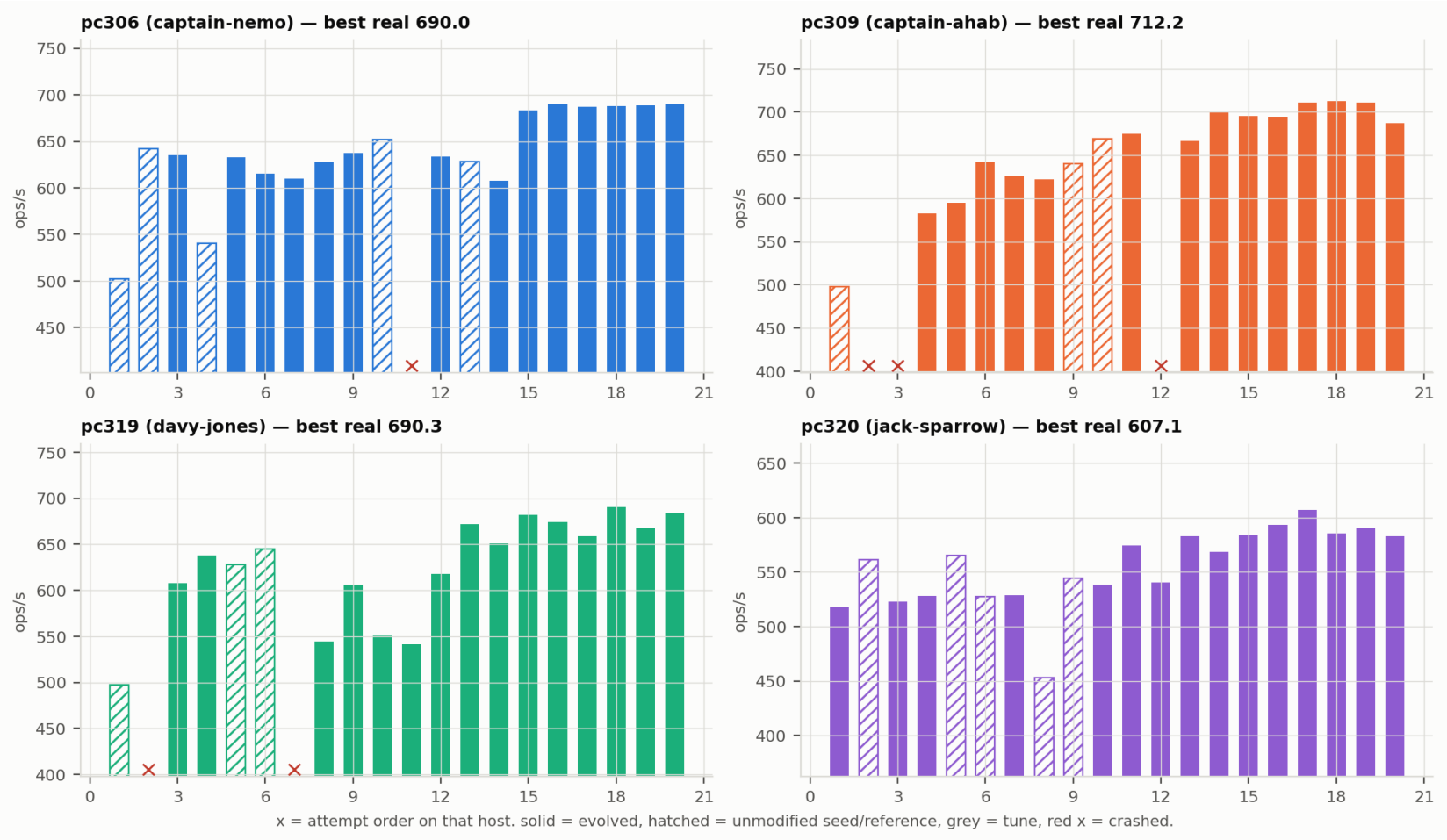
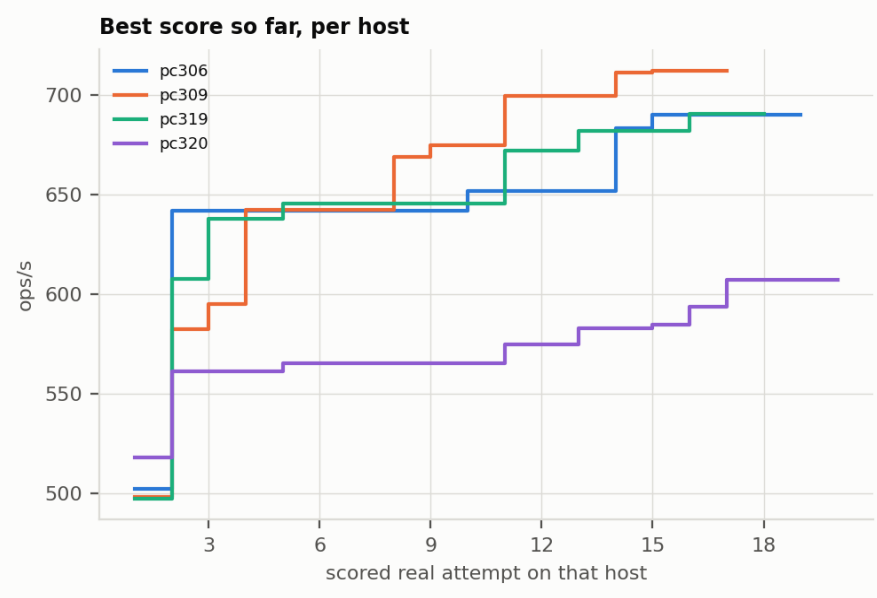
Multi-host CORAL evolution — run 2026-10-21_003256

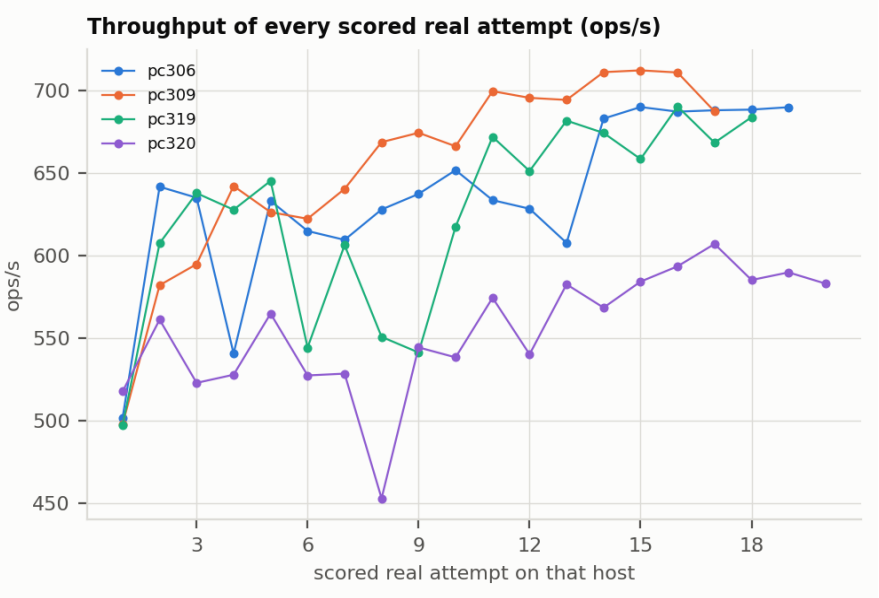
4 agents, one per CloudLab host (each agent evaluated only on its own host, in parallel); score = absolute measured throughput (ops/s). Budget 20 real evaluations per host; all attempts are in one shared leaderboard.

host	agent	real / tune / crashed	best score	its tput (ops/s)	max tput (ops/s)	host no-op baseline (ops/s, n=8; tanh only)	LLM cost
pc306	captain-nemo	20 / 0 / 1	#56 3f1573c8 690.0	690.0	690.0	-	$6
pc309	captain-ahab	20 / 0 / 3	#78 7e6df4dc 712.2	712.2	712.2	-	$7
pc319	davy-jones	20 / 0 / 2	#65 50b9f179 690.3	690.3	690.3	-	$10
pc320	jack-sparrow	20 / 0 / 0	#68 6b62bff1 607.1	607.1	607.1	-	$9

- Overall best: pc309 (captain-ahab), #78 7e6df4dc, score 712.2, 712.2 ops/s — agent-evolved.
- Run stop: max_real_attempts; wall time 1.02 h; total LLM cost $32; gateway requests {400: 174, 200: 886, 503: 5}.
- Hosts differ in speed, so raw throughput is not comparable across hosts: compare changes within a host, not levels across hosts.







Top 10 attempts across all hosts (real, scored)

#	host	agent	attempt	commit	score	tput	MiB read	refaults	design / title
1	pc309	captain-ahab	78	7e6df4dc	712.2	712.2	192	23486	repeat of 0a187508 (region-CLOCK + region-coherent eviction, 711.2) unchanged except comment, to confirm +2.2% over same-host cont
2	pc309	captain-ahab	77	0a187508	711.1	711.1	190	23035	region-CLOCK 50b9f179 + region-coherent eviction (jack-sparrow draft, last untested skill extension): evicting a folio dooms its 3
3	pc309	captain-ahab	79	db1edf33	710.9	710.9	194	24137	region-coherent eviction DOOM_NS 100ms -> 300ms (base 7e6df4dc confirmed 711.2/712.2 on pc309). Longer doom window catches region-
4	pc309	captain-ahab	72	c56fed27	699.6	699.6	184	21856	region-CLOCK team-best 50b9f179 (690 pc319) + miss-cost weighting: GET-thread accesses +2, scan-thread +1 (scan_pids reused from h
5	pc309	captain-ahab	75	acb7b208	695.6	695.6	188	22778	same-host control: team-best 50b9f179 (region-CLOCK) unchanged (comment only) on pc309, to attribute my c56fed27 GET x2 result (69
6	pc309	captain-ahab	76	32f8132e	694.3	694.3	189	22832	region-CLOCK 50b9f179 with counter array 2^19 -> 2^21 slots (fewer region hash collisions inflating cold-region counts). Same-host
7	pc319	davy-jones	65	50b9f179	690.3	690.3	189	22768	region-freq CLOCK: nemo combo 51067b5b (shift3 + grace 200ms, 687 pc306) + DEDUPE 2ms->20ms (region count = number of distinct 20m
8	pc306	captain-nemo	56	3f1573c8	690.0	690.0	196	24377	region-CLOCK grace sweep 1/2: davy-jones f578a1ba (683 on pc306) with GRACE_NS 1s -> 200ms. Motivation: mglru head- vs tail-insert
9	pc306	captain-nemo	69	3d305d1f	689.9	689.9	196	24495	variance repeat (final eval 20/20): 3f1573c8 unchanged except comment (region-CLOCK shift4, grace 200ms; first run 690.0). pc306 r
10	pc306	captain-nemo	66	b9876718	688.5	688.5	195	24236	region-CLOCK DEDUPE_NS 2ms -> 20ms (untaken knob) on my best 3f1573c8 (shift4, grace 200ms, 690). Region counter aggregates 16 pag

All attempts on pc306 (captain-nemo)

#	commit	class	score	tput	read MiB	refaults	wall s	design / title
1	eb8d23ba	real =get_scan	501.9	501.9	136	11307	40	baseline: reference/get_scan.c unchanged (scan-pid list + sampled LFU w/ index-block bonus) to calibrate my host
2	bceae5c4	real =s3fifo	641.8	641.8	175	19462	41	baseline 2: reference/s3fifo.c unchanged. Context: get_scan.c scored 502 (2.4x below noop ~1215) on pc306; hypothesis H1 = get_scan proposes
3	b17a7960	real	635.1	635.1	175	19573	41	s3fifo + LevelDB index-block pinning: last page of each file admitted straight to main w/ freq 3 and skipped by main CLOCK. Base: my s3fifo
4	9a36aaf7	real =sampling	540.8	540.8	146	12995	41	baseline 3: reference/sampling.c unchanged (sampled LFU k=20 + index bonus). Question: does pure frequency beat s3fifo (642) on Zipfian GETs
5	daeac2fb	real	633.2	633.2	175	19649	41	s3fifo knob 1/3: small queue 25% of cache (ref: cache/15 or main<=2*small). Hypothesis: 800-page small window too short for warm Zipfian GET
6	4ddceb13	real	614.9	614.9	174	19341	41	structural 1/3 readahead-aware s3fifo: eager promotion to main in folio_accessed (list_move) + never-accessed small folios (freq 0, e.g. unc
7	44700168	real	609.6	609.6	172	18938	40	structural 2/3 access-aware s3fifo: INVERT eval#5's lap rule. Eval#5 (lap for freq-0) = 615, bytes/IO unchanged (15.5KB) -> readahead-thrash
8	5a7382d2	real	628.0	628.0	174	19289	41	structural 3/3 ablation: reference s3fifo + ONLY eager promotion (list_move to main in folio_accessed on 2nd hit). Evals #5/#6 (both w/ eage
9	1ff547d4	real	637.4	637.4	176	19584	41	s3fifo ghost 4x cache (ref 1x). Rationale: LevelDB block cache above page cache makes page-cache reuse distances long (exclusive-cache view,
10	b75f5c54	real =mglru	651.8	651.8	193	23772	41	reviewer: reference/mglru.c unchanged on pc306 to resolve 645 (davy) vs 565 (jack) cross-host split for identical code. My s3fifo-family bas
11	a5bbf501	real ✗	crash	-	0	0	3	eval: [FAIL] wall=3.49s error=benchmark[all] returned 1
12	7750f4f9	real	633.6	633.6	188	22815	41	structural 1/3 (retry, verifier fix) mglru+ghost-admission: refaulting folios added to youngest gen. Prior eval failed verifier: ghost tier
13	0c4616da	real =noop	628.4	628.4	179	20410	41	reviewer: unmodified noop.c on pc306 to anchor my host. ahab: noop=640 on pc309, jack: 561 on pc320 -> hosts differ ~14%. My pc306: mglru 65
14	53c00e6d	real	607.7	607.7	190	23048	41	mglru lane 2/3: reference mglru (652; noop on pc306 = 628) with new folios added at TAIL of gen list instead of head. Ref list_add puts new
15	05ad47b0	real	683.0	683.0	197	24570	40	reviewer replication: davy-jones f578a1ba region-frequency CLOCK (672 on pc319) UNCHANGED on pc306. My pc306 anchors: noop 628, mglru 652, s
16	3f1573c8	real	690.0	690.0	196	24377	41	region-CLOCK grace sweep 1/2: davy-jones f578a1ba (683 on pc306) with GRACE_NS 1s -> 200ms. Motivation: mglru head- vs tail-insertion (652 v
17	51067b5b	real	687.2	687.2	193	23633	41	region-CLOCK combo: GRACE_NS 200ms (mine, 690 on pc306 vs 683 at 1s) + REGION_SHIFT 3 (davy-jones best on pc319: 682 vs 672 at shift 4).
18	4173866b	real	688.0	688.0	197	24559	40	region-CLOCK grace sweep 2/2: GRACE_NS 50ms, REGION_SHIFT 4. pc306 so far: grace 1s=683, 200ms=690, 200ms+shift3=687 (all within noise). Tes
19	b9876718	real	688.5	688.5	195	24236	41	region-CLOCK DEDUPE_NS 2ms -> 20ms (untaken knob) on my best 3f1573c8 (shift4, grace 200ms, 690). Region counter aggregates 16 pages; a scan
20	3d305d1f	real	689.9	689.9	196	24495	41	variance repeat (final eval 20/20): 3f1573c8 unchanged except comment (region-CLOCK shift4, grace 200ms; first run 690.0). pc306 region-CLOC

All attempts on pc309 (captain-ahab)

#	commit	class	score	tput	read MiB	refault s	wall s	design / title
1	ec1f1584	real =get_scan	497.9	497.9	133	10651	41	baseline: reference get_scan.c unchanged (scan-pid list + sampled LFU) to calibrate my host
2	86f88956	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
3	848c1254	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
4	0b0faad4	real	582.1	582.1	157	15608	41	structural 1/3 scan-aware S3-FIFO (compile fix 2: static size counters): scan-pid list evicted first (256-page reserve), small FIFO 10% + CL
5	a7c5ead2	real	594.9	594.9	159	16080	40	scan-aware S3-FIFO v2 (2/3): ghost ALL non-scan evictions (incl MAIN), ghost 64k entries storing freq; ghost hit -> MAIN with freq=min(2,1+g
6	f4670f97	real	642.2	642.2	173	19144	41	ablation 3/3: v2 with scan-pid special casing OFF (no scan list, scan accesses count) -> plain S3-FIFO + ghost-all w/ freq. Tests hypothesis
7	c142f84a	real	626.3	626.3	166	17675	40	structural 1/3 NEW-filtered S3-FIFO: never-accessed folios (GET readahead junk) in NEW list evicted first beyond 512-page reserve; first acc
8	fe40227e	real	622.4	622.4	172	18866	41	long-memory ghost: f4670f97 (642) with ghost 64k per-CPU-LRU -> 256k common LRU (~20x cache). Two-level-cache view: page-cache reuse distanc
9	ad992e58	real =noop	640.4	640.4	175	19379	41	calibration on fast host pc309: unmodified noop.c (kernel default, comment-only change). Open question: is kernel default ~640 on fast hosts
10	21e921d8	real =mglru	668.8	668.8	187	22572	40	reviewer: same-host pair. reference/mglru.c unchanged on pc309, where noop = 640.4 (ad992e58). Other hosts: mglru 645 (pc319), 652 (nemo). T
11	0a062399	real	674.4	674.4	193	24080	41	mglru + access aging: halve folio access count (min 1) on each tier-protection promotion, like kernel folio_inc_gen clearing refs. Ref mglru
12	3b29baee	real ✗	crash	-	0	0	4	eval: [FAIL] wall=3.77s error=benchmark[all] returned 1
13	55e53c70	real	666.2	666.2	193	24045	41	structural 1/3 (verifier fix: hash i_ino not host ptr) MGLRU + persistent region tiers: tier from folio accesses + decayed 64KB-region count
14	c56fed27	real	699.6	699.6	184	21856	40	region-CLOCK team-best 50b9f179 (690 pc319) + miss-cost weighting: GET-thread accesses +2, scan-thread +1 (scan_pids reused from harness pin
15	acb7b208	real	695.6	695.6	188	22778	40	same-host control: team-best 50b9f179 (region-CLOCK) unchanged (comment only) on pc309, to attribute my c56fed27 GET x2 result (699.6, same
16	32f8132e	real	694.3	694.3	189	22832	41	region-CLOCK 50b9f179 with counter array 2^19 -> 2^21 slots (fewer region hash collisions inflating cold-region counts). Same-host control a
17	0a187508	real	711.1	711.1	190	23035	40	region-CLOCK 50b9f179 + region-coherent eviction (jack-sparrow draft, last untested skill extension): evicting a folio dooms its 32KB region
18	7e6df4dc	real	712.2	712.2	192	23486	41	repeat of 0a187508 (region-CLOCK + region-coherent eviction, 711.2) unchanged except comment, to confirm +2.2% over same-host control 695.6
19	db1edf33	real	710.9	710.9	194	24137	41	region-coherent eviction DOOM_NS 100ms -> 300ms (base 7e6df4dc confirmed 711.2/712.2 on pc309). Longer doom window catches region-mates furt
20	10ecff9e	real	687.3	687.3	196	24532	41	final eval 20/20: region-coherent eviction (best 7e6df4dc, 711/712) with REGION_SHIFT 3 -> 4. Without doom shift 4 lost (672 vs 682 pc319);

All attempts on pc319 (davy-jones)

#	commit	class	score	tput	read MiB	refault s	wall s	design / title
1	f3034a35	real =get_scan	497.3	497.3	133	10674	41	baseline: reference/get_scan.c unchanged (scan-pid list + sampled LFU) to calibrate my host
2	c478eaf0	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
3	3c5a7dfe	real	607.5	607.5	164	16998	41	scan-aware S3-FIFO (fixed ctx compile err): scan-pid folios -> scan list evicted first (promote to main if GET-touched); GET folios -> small
4	0a4d0eed	real	637.9	637.9	178	20068	41	falsification A/B on s3fifo ref (642): scan-pid-inserted folios admitted straight to main (opposite of scan-first eviction). Tests hypothesi
5	77a39eb0	real =fifo	627.8	627.8	190	23105	41	diagnostic: reference/fifo.c unchanged (no accessed hook, no maps). Bytes/op is ~9.1-9.6KB across ALL attempts (miss rate ~policy-invariant)
6	57af07f9	real =mglru	645.3	645.3	196	24564	40	reference/mglru.c unchanged (untested reference). Context: fifo 628 ~= s3fifo 642 > noop 561 -> hit-ratio quality weakly matters; rios ~10-1
7	8f1fdb1a	real ✗	crash	-	-	-	-	eval: compile failed: compile failed
8	925ce075	real	544.4	544.4	144	12673	41	structural 1/3 history-LFU (retry; prev fail = 'struct slot' clashes with vmlinux.h, renamed): single list, sampled(16) min-freq eviction, f
9	7ff71cd0	real	606.4	606.4	168	17956	41	s3fifo + consumed-scan-first: scan-inserted folios move to a consumed list on the scan's first READ and that list is evicted first (GET-touc
10	281d22ad	real	550.9	550.9	145	12835	41	structural 2/3 history-LFU: v1 (544, 925ce075) + count at most one access per page per 2ms (scan thread reads many keys/page in a burst -> v
11	d04239a3	real	541.4	541.4	146	13090	40	structural 3/3 history-LFU: v2 (551) + 1s grace for newly inserted folios (score 2^40+count). Hypothesis: throughput tracks bytes/disk-IO (r
12	f2ca3e81	real	617.5	617.5	170	18377	41	history-LFU via ITERATE not SAMPLE: same persistent freq sketch + 1s grace as 541-551 versions, but eviction = CLOCK threshold passes (count
13	f578a1ba	real	672.0	672.0	195	24196	40	structural 1/3 region-frequency CLOCK: f2ca3e81 (617, history-LFU via iterate) with the persistent counter keyed by (ino, index>>4) = 64KB r
14	38ee907f	real	651.3	651.3	187	22443	40	structural 2/3 region-frequency CLOCK: REGION_SHIFT 4->5 (128KB regions = default readahead window). Parent f578a1ba = 672
15	75eae97c	real	681.8	681.8	192	23482	41	structural 3/3 region-frequency CLOCK: REGION_SHIFT 3 (32KB). Sweep so far: shift4(64KB)=672 f578a1ba, shift5(128KB)=651 38ee907f, page-leve
16	2edd0685	real	674.3	674.3	188	22668	41	region-frequency CLOCK sweep: REGION_SHIFT 2 (16KB). So far shift0 617, shift3 682, shift4 672, shift5 651 (pc319)
17	7947fa05	real	658.5	658.5	191	23318	41	region-freq CLOCK shift3 (682) + CLOCK pass thresholds 1/3/8 -> 2/6/16 (region counters aggregate 8 pages so counts run higher)
18	50b9f179	real	690.3	690.3	189	22768	41	region-freq CLOCK: nemo combo 51067b5b (shift3 + grace 200ms, 687 pc306) + DEDUPE 2ms->20ms (region count = number of distinct 20ms windows
19	2e5d5053	real	668.4	668.4	186	22226	41	region-freq CLOCK 50b9f179 (690) with LOWER CLOCK thresholds 1/3/8 -> 0/1/3 (raising to 2/6/16 lost 682->658, so test the other direction)
20	6a2eaff5	real	683.9	683.9	188	22531	40	final: resubmit of 50b9f179 (region-freq CLOCK, 690.3) with a comment-only change, to measure same-host run-to-run variance

All attempts on pc320 (jack-sparrow)

#	commit	class	score	tput	read MiB	refaults	wall s	design / title
1	48ab7261	real	517.8	517.8	146	13194	41	v1 structural 1/3: scan-first FIFO drain + CLOCK(2-bit freq) general list, promote scan pages touched by GET threads, index-page protection
2	897780ff	real =noop	561.3	561.3	162	16723	41	calibration: unmodified noop.c (kernel default reclaim) on my host, to check whether ~500 ops/s BPF policies are below the ~1215 noop refere
3	392a37e1	real	523.0	523.0	148	13700	41	v2: scan-only minimal-overhead policy (no per-folio map, no folio_accessed); evict scan-pid pages FIFO first, propose nothing else -> kernel
4	8cd537d2	real	527.9	527.9	153	14888	41	v3: v2 scan-only with SCAN_MIN_PAGES 32->2048 (8MiB reserve). Tests readahead-thrash hypothesis: rios ~constant across all policies, score t
5	212162dd	real =mglru	564.9	564.9	168	18230	41	lane change (scan-first lane closed: reserve 32->2048 gave 523->528, all scan-first policies < noop 561 < s3fifo 641; scan pages appear reus
6	30244447	real =mru	527.5	527.5	158	16516	41	hypothesis test: LevelDB block cache sits above page cache, so page-cache hit signal is inverted (2nd-level cache). Evidence: LFU 498-540 <
7	123b7e87	real	528.6	528.6	174	19839	41	falsification (mirror of my scan-first v3=528): GET-FIRST two-FIFO - GET pages evicted first beyond 2048-page reserve, scan-pid pages protec
8	c63ebbfd	real =lhd	453.1	453.1	125	9092	41	HOST CALIBRATION WARNING: pc320 (mine) is ~12% slower than pc319: mglru.c = 565 here vs 645 on davy's pc319; my noop 561 => kernel default ~
9	c2899433	real =s3fifo	544.5	544.5	154	14913	41	REVIEW/falsification: does s3fifo actually beat kernel default? Only I have measured noop (561 on pc320); team compares s3fifo 641 (pc306) t
10	20218501	real	538.4	538.4	163	16951	40	mglru-tune 1/3 (engineer lane: MGLRU is best within-host on pc306 651.8 vs s3fifo 641.8 and on pc320 565 vs s3fifo 544). Ablation: insert ne
11	e61194f0	real	574.5	574.5	166	17934	40	mglru-tune 2/3: tune1 (tail insert) LOST 565->538, so evicting newest-inserted never-accessed folios (accesses==1, i.e. unconsumed/unused re
12	7a6652e4	real	540.3	540.3	163	17176	41	mglru-tune 3/3: tune2 (always-oldest-gen insert, 574.5, my best vs ref 565) + refaulted folios whose ghost tier>=1 (accessed before eviction
13	10e9a948	real	582.7	582.7	172	18986	41	cross-host replication: davy-jones f578a1ba region-frequency CLOCK (672 on pc319) unchanged (comment only) on pc320. pc320 refs: mglru 565,
14	e5fa0ac1	real	568.5	568.5	164	17375	41	region-freq CLOCK (davy f578a1ba; replicated 582.7 on pc320 = +3.1% over mglru 565) + grace only when region count>0: never-accessed (prefet
15	2a05859b	real	584.3	584.3	170	18481	41	region-freq CLOCK knob nobody is sweeping: AGE_SHIFT 18->15 (8x faster count halving; at 2^18 counts barely decay within a 30s run). Base da
16	922e509b	real	593.6	593.6	169	18372	41	cross-host replication of team-best config: captain-nemo 51067b5b (region-freq CLOCK, REGION_SHIFT 3, GRACE 200ms; 687 on pc306) unchanged o
17	6b62bff1	real	607.1	607.1	169	18405	41	region-freq CLOCK pass thresholds (1,3,8) -> (2,4,16) on team-best nemo 51067b5b (593.6 on pc320). Untouched knob: first pass now also evict
18	9392a608	real	585.3	585.3	170	18651	41	thresholds trend test: (2,4,16)=607.1 > (1,3,8)=593.6 on pc320; now (3,6,24) on nemo 51067b5b base
19	e7b0c611	real	589.9	589.9	167	17944	41	confirmation re-run of 6b62bff1 thresholds (2,4,16) (607.1). Neighbors on pc320: (1,3,8)=593.6, (3,6,24)=585.3
20	64eef3e1	real	583.1	583.1	164	17132	41	final eval 20/20: cross-host replication of team best captain-ahab c56fed27 (699.6 pc309; region-CLOCK + GET-access weight 2) unchanged on p

pc309 (captain-ahab): best policy #78 7e6df4dc — score 712.2, 712.2 ops/s — overall best

Agent's title: repeat of 0a187508 (region-CLOCK + region-coherent eviction, 711.2) unchanged except comment, to confirm +2.2% over same-host control 695.6

Closest seed by line similarity: fifo 74%, s3fifo 51%, mru 45%. Lines that differ from fifo are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// [captain-nemo: davy-jones f578a1ba with GRACE_NS 1s -> 200ms, REGION_SHIFT 4 -> 3]
// [captain-ahab: region-coherent eviction (draft by jack-sparrow): once a folio of a region is
//  evicted, its region-mates are victims for DOOM_NS too, even inside grace; a re-read clears it]
// [repeat run]
// History-LFU (davy-jones): one list, CLOCK-style threshold passes (iterate API) evicting
// low-frequency folios.
// Frequency lives in a fixed-size counter array keyed by hash(inode, page index), so it
// SURVIVES eviction. Accesses to the same page within 2ms count once (the scan thread
// reads many keys per page in a burst, which otherwise pins scan pages): a hot page that is evicted and re-read keeps its count, unlike a
// per-folio map. No per-folio metadata map at all. Counters are halved every AGE_PERIOD
// accesses (lazy, via an epoch stamp per slot) so the history slowly forgets.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define SKETCH_BITS 19
#define REGION_SHIFT 3      /* 8 pages = 32KB regions */
#define SKETCH_SIZE (1 << SKETCH_BITS)
#define SKETCH_MASK (SKETCH_SIZE - 1)
#define AGE_SHIFT 18        /* epoch advances every 2^18 accesses */
#define SAMPLE_SIZE 16
#define FREQ_MAX 0xffff
#define SCORE_MAX (0x7fffffffffffffffLL)

#define DEDUPE_NS (20ULL * 1000 * 1000) /* bursts within 20ms count once */

struct dj_freq_slot {
    u32 count;
    u32 epoch;
    u64 last_ns;
    u64 added_ns;
    u64 doomed_ns;
};

#define DOOM_NS (100ULL * 1000 * 1000)  /* region-mates of a victim are victims for 100ms */

#define GRACE_NS (200ULL * 1000 * 1000) /* new folios are not victims for 200ms */

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, u32);
    __type(value, struct dj_freq_slot);
    __uint(max_entries, SKETCH_SIZE);
} sketch SEC(".maps");

static u64 main_list;
static u64 nr_accesses = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline u32 folio_slot(struct folio *folio)
{
    u64 ino = folio->mapping->host->i_ino;
    // Region granularity: all pages of a 2^REGION_SHIFT-page file region share one counter,
    // so a readahead batch is kept or evicted as a unit and refaults come back as one IO.
    u64 region = (u64)folio->index >> REGION_SHIFT;
    u64 h = (ino * 0x9E3779B97F4A7C15ULL) ^ (region * 0xC2B2AE3D27D4EB4FULL);
    h ^= h >> 29;
    return (u32)(h & SKETCH_MASK);
}

static inline struct dj_freq_slot *get_slot(struct folio *folio, u32 epoch)
{
    u32 key = folio_slot(folio);
    struct dj_freq_slot *s = bpf_map_lookup_elem(&sketch, &key);
    if (!s)
        return NULL;
    u32 old = s->epoch;
    if (old != epoch) {
        u32 d = epoch - old;
        s->count = d >= 16 ? 0 : (s->count >> d);
        s->epoch = epoch;
    }
    return s;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    return 0;
}

static inline s64 folio_count(struct folio *f)
{
    if (!f->mapping || !f->mapping->host)
        return 0;
    struct dj_freq_slot *s = get_slot(f, (u32)(nr_accesses >> AGE_SHIFT));
    if (!s)
        return 0;
    u64 now = bpf_ktime_get_ns();
    if (now - s->doomed_ns < DOOM_NS)
        return -1;
    if (now - s->added_ns < GRACE_NS)
        return 1LL << 40;
    return s->count;
}

static inline void doom_region(struct folio *f)
{
    if (!f->mapping || !f->mapping->host)
        return;
    struct dj_freq_slot *s = get_slot(f, (u32)(nr_accesses >> AGE_SHIFT));
    if (s)
        s->doomed_ns = bpf_ktime_get_ns();
}

// CLOCK-style pass over the list: evict folios whose (persistent) count <= thr, move the
// rest to the tail. Uses list_iterate instead of list_sample (every sample-based policy
// on the team scores 500-550, every iterate-based one >= 600).
#define LFU_ITER_FN(thr)                                \
static int lfu_iter_fn_##thr(int idx, struct cache_ext_list_node *a)            \
{                                                       \
    struct folio *f = a->folio;                         \
    if (!folio_test_uptodate(f) || !folio_test_lru(f))              \
        return CACHE_EXT_CONTINUE_ITER;                     \
    if (folio_test_dirty(f) || folio_test_writeback(f))             \
        return CACHE_EXT_CONTINUE_ITER;                     \
    if (folio_count(f) <= thr) {                            \
        doom_region(f);                                 \
        return CACHE_EXT_EVICT_NODE;                        \
    }                                                   \
    return CACHE_EXT_CONTINUE_ITER;                         \
}

LFU_ITER_FN(1)
LFU_ITER_FN(3)
LFU_ITER_FN(8)

static int any_iter_fn(int idx, struct cache_ext_list_node *a)
{
    struct folio *f = a->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
```

```
            struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_1, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_3, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_8, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate_extended(memcg, main_list, any_iter_fn, &opts, eviction_ctx);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    u64 n = __sync_fetch_and_add(&nr_accesses, 1);
    struct dj_freq_slot *s = get_slot(folio, (u32)(n >> AGE_SHIFT));
    if (!s)
        return;
    u64 now = bpf_ktime_get_ns();
    if (now - s->last_ns < DEDUPE_NS)
        return;
    s->last_ns = now;
    s->doomed_ns = 0;
    if (s->count < FREQ_MAX)
        s->count += 1;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    if (bpf_cache_ext_list_add_tail(main_list, folio)) {
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }
    struct dj_freq_slot *s = get_slot(folio, (u32)(nr_accesses >> AGE_SHIFT));
    if (s) {
        s->added_ns = bpf_ktime_get_ns();
        s->doomed_ns = 0;   /* refault into a doomed region: it is wanted after all */
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 },
};

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
```

```
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }
    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        goto cleanup;
    }

    printf("evo_policy (FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc306 (captain-nemo): best policy #56 3f1573c8 — score 690.0, 690.0 ops/s

Agent's title: region-CLOCK grace sweep 1/2: davy-jones f578a1ba (683 on pc306) with GRACE_NS 1s -> 200ms. Motivation: mglru head- vs tail-insertion (652 vs 608) says evicting un-re-referenced new pages early helps; 1s grace may over-protect readahead/one-shot regions.

Closest seed by line similarity: fifo 77%, s3fifo 52%, mru 46%. Lines that differ from fifo are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// [captain-nemo: davy-jones f578a1ba with GRACE_NS 1s -> 200ms]
// History-LFU (davy-jones): one list, CLOCK-style threshold passes (iterate API) evicting
// low-frequency folios.
// Frequency lives in a fixed-size counter array keyed by hash(inode, page index), so it
// SURVIVES eviction. Accesses to the same page within 2ms count once (the scan thread
// reads many keys per page in a burst, which otherwise pins scan pages): a hot page that is evicted and re-read keeps
// per-folio map. No per-folio metadata map at all. Counters are halved every AGE_PERIOD
// accesses (lazy, via an epoch stamp per slot) so the history slowly forgets.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define SKETCH_BITS 19
#define REGION_SHIFT 4      /* 16 pages = 64KB regions */
#define SKETCH_SIZE (1 << SKETCH_BITS)
#define SKETCH_MASK (SKETCH_SIZE - 1)
#define AGE_SHIFT 18        /* epoch advances every 2^18 accesses */
#define SAMPLE_SIZE 16
#define FREQ_MAX 0xffff
#define SCORE_MAX (0x7fffffffffffffffLL)

#define DEDUPE_NS (2ULL * 1000 * 1000)  /* bursts within 2ms count once */

struct dj_freq_slot {
    u32 count;
    u32 epoch;
    u64 last_ns;
    u64 added_ns;
};

#define GRACE_NS (200ULL * 1000 * 1000) /* new folios are not victims for 200ms */

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, u32);
    __type(value, struct dj_freq_slot);
    __uint(max_entries, SKETCH_SIZE);
} sketch SEC(".maps");

static u64 main_list;
static u64 nr_accesses = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline u32 folio_slot(struct folio *folio)
{
    u64 ino = folio->mapping->host->i_ino;
    // Region granularity: all pages of a 2^REGION_SHIFT-page file region share one counter,
    // so a readahead batch is kept or evicted as a unit and refaults come back as one IO.
    u64 region = (u64)folio->index >> REGION_SHIFT;
    u64 h = (ino * 0x9E3779B97F4A7C15ULL) ^ (region * 0xC2B2AE3D27D4EB4FULL);
    h ^= h >> 29;
    return (u32)(h & SKETCH_MASK);
}

static inline struct dj_freq_slot *get_slot(struct folio *folio, u32 epoch)
{
    u32 key = folio_slot(folio);
    struct dj_freq_slot *s = bpf_map_lookup_elem(&sketch, &key);
    if (!s)
        return NULL;
    u32 old = s->epoch;
    if (old != epoch) {
        u32 d = epoch - old;
        s->count = d >= 16 ? 0 : (s->count >> d);
        s->epoch = epoch;
    }
    return s;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    return 0;
}

static inline s64 folio_count(struct folio *f)
{
    if (!f->mapping || !f->mapping->host)
        return 0;
    struct dj_freq_slot *s = get_slot(f, (u32)(nr_accesses >> AGE_SHIFT));
    if (!s)
        return 0;
    if (bpf_ktime_get_ns() - s->added_ns < GRACE_NS)
        return 1LL << 40;
    return s->count;
}

// CLOCK-style pass over the list: evict folios whose (persistent) count <= thr, move the
// rest to the tail. Uses list_iterate instead of list_sample (every sample-based policy
// on the team scores 500-550, every iterate-based one >= 600).
#define LFU_ITER_FN(thr)                                        \
static int lfu_iter_fn_##thr(int idx, struct cache_ext_list_node *a)            \
{                                                               \
    struct folio *f = a->folio;                                 \
    if (!folio_test_uptodate(f) || !folio_test_lru(f))              \
        return CACHE_EXT_CONTINUE_ITER;                         \
    if (folio_test_dirty(f) || folio_test_writeback(f))             \
        return CACHE_EXT_CONTINUE_ITER;                         \
    if (folio_count(f) <= thr)                                  \
        return CACHE_EXT_EVICT_NODE;                            \
    return CACHE_EXT_CONTINUE_ITER;                             \
}

LFU_ITER_FN(1)
LFU_ITER_FN(3)
LFU_ITER_FN(8)

static int any_iter_fn(int idx, struct cache_ext_list_node *a)
{
    struct folio *f = a->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_1, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_3, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_8, &opts, eviction_ctx) < 0)
```

```
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate_extended(memcg, main_list, any_iter_fn, &opts, eviction_ctx);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    u64 n = __sync_fetch_and_add(&nr_accesses, 1);
    struct dj_freq_slot *s = get_slot(folio, (u32)(n >> AGE_SHIFT));
    if (!s)
        return;
    u64 now = bpf_ktime_get_ns();
    if (now - s->last_ns < DEDUPE_NS)
        return;
    s->last_ns = now;
    if (s->count < FREQ_MAX)
        s->count += 1;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    if (bpf_cache_ext_list_add_tail(main_list, folio)) {
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }
    struct dj_freq_slot *s = get_slot(folio, (u32)(nr_accesses >> AGE_SHIFT));
    if (s)
        s->added_ns = bpf_ktime_get_ns();
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 },
};

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;
```

```
    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }
    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        goto cleanup;
    }

    printf("evo_policy (FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc319 (davy-jones): best policy #65 50b9f179 — score 690.3, 690.3 ops/s

Agent's title: region-freq CLOCK: nemo combo 51067b5b (shift3 + grace 200ms, 687 pc306) + DEDUPE 2ms->20ms (region count = number of distinct 20ms windows with an access; less burst-inflated). Untouched knob

Closest seed by line similarity: fifo 77%, s3fifo 52%, mru 46%. Lines that differ from fifo are highlighted.

```
// ================================================================================
// SECTION: BPF KERNEL CODE
// ================================================================================
// EVOLVE-BLOCK-START
// [captain-nemo: davy-jones f578a1ba with GRACE_NS 1s -> 200ms, REGION_SHIFT 4 -> 3]
// History-LFU (davy-jones): one list, CLOCK-style threshold passes (iterate API) evicting
// low-frequency folios.
// Frequency lives in a fixed-size counter array keyed by hash(inode, page index), so it
// SURVIVES eviction. Accesses to the same page within 2ms count once (the scan thread
// reads many keys per page in a burst, which otherwise pins scan pages): a hot page that is evicted and re-read keeps its count, unlike a
// per-folio map. No per-folio metadata map at all. Counters are halved every AGE_PERIOD
// accesses (lazy, via an epoch stamp per slot) so the history slowly forgets.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define SKETCH_BITS 19
#define REGION_SHIFT 3      /* 8 pages = 32KB regions */
#define SKETCH_SIZE (1 << SKETCH_BITS)
#define SKETCH_MASK (SKETCH_SIZE - 1)
#define AGE_SHIFT 18        /* epoch advances every 2^18 accesses */
#define SAMPLE_SIZE 16
#define FREQ_MAX 0xffff
#define SCORE_MAX (0x7fffffffffffffffLL)

#define DEDUPE_NS (20ULL * 1000 * 1000) /* bursts within 20ms count once */

struct dj_freq_slot {
    u32 count;
    u32 epoch;
    u64 last_ns;
    u64 added_ns;
};

#define GRACE_NS (200ULL * 1000 * 1000) /* new folios are not victims for 200ms */

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, u32);
    __type(value, struct dj_freq_slot);
    __uint(max_entries, SKETCH_SIZE);
} sketch SEC(".maps");

static u64 main_list;
static u64 nr_accesses = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline u32 folio_slot(struct folio *folio)
{
    u64 ino = folio->mapping->host->i_ino;
    // Region granularity: all pages of a 2^REGION_SHIFT-page file region share one counter,
    // so a readahead batch is kept or evicted as a unit and refaults come back as one IO.
    u64 region = (u64)folio->index >> REGION_SHIFT;
    u64 h = (ino * 0x9E3779B97F4A7C15ULL) ^ (region * 0xC2B2AE3D27D4EB4FULL);
    h ^= h >> 29;
    return (u32)(h & SKETCH_MASK);
}

static inline struct dj_freq_slot *get_slot(struct folio *folio, u32 epoch)
{
    u32 key = folio_slot(folio);
    struct dj_freq_slot *s = bpf_map_lookup_elem(&sketch, &key);
    if (!s)
        return NULL;
    u32 old = s->epoch;
    if (old != epoch) {
        u32 d = epoch - old;
        s->count = d >= 16 ? 0 : (s->count >> d);
        s->epoch = epoch;
    }
    return s;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    return 0;
}

static inline s64 folio_count(struct folio *f)
{
    if (!f->mapping || !f->mapping->host)
        return 0;
    struct dj_freq_slot *s = get_slot(f, (u32)(nr_accesses >> AGE_SHIFT));
    if (!s)
        return 0;
    if (bpf_ktime_get_ns() - s->added_ns < GRACE_NS)
        return 1LL << 40;
    return s->count;
}

// CLOCK-style pass over the list: evict folios whose (persistent) count <= thr, move the
// rest to the tail. Uses list_iterate instead of list_sample (every sample-based policy
// on the team scores 500-550, every iterate-based one >= 600).
#define LFU_ITER_FN(thr)                                        \
static int lfu_iter_fn_##thr(int idx, struct cache_ext_list_node *a)            \
{                                                               \
    struct folio *f = a->folio;                                 \
    if (!folio_test_uptodate(f) || !folio_test_lru(f))              \
        return CACHE_EXT_CONTINUE_ITER;                         \
    if (folio_test_dirty(f) || folio_test_writeback(f))             \
        return CACHE_EXT_CONTINUE_ITER;                         \
    if (folio_count(f) <= thr)                                  \
        return CACHE EXT EVICT NODE;                            \
    return CACHE_EXT_CONTINUE_ITER;                             \
}

LFU_ITER_FN(1)
LFU_ITER_FN(3)
LFU_ITER_FN(8)

static int any_iter_fn(int idx, struct cache_ext_list_node *a)
{
    struct folio *f = a->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_1, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_3, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_8, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
```

```
        return;
    bpf_cache_ext_list_iterate_extended(memcg, main_list, any_iter_fn, &opts, eviction_ctx);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    u64 n = __sync_fetch_and_add(&nr_accesses, 1);
    struct dj_freq_slot *s = get_slot(folio, (u32)(n >> AGE_SHIFT));
    if (!s)
        return;
    u64 now = bpf_ktime_get_ns();
    if (now - s->last_ns < DEDUPE_NS)
        return;
    s->last_ns = now;
    if (s->count < FREQ_MAX)
        s->count += 1;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    if (bpf_cache_ext_list_add_tail(main_list, folio)) {
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }
    struct dj_freq_slot *s = get_slot(folio, (u32)(nr_accesses >> AGE_SHIFT));
    if (s)
        s->added_ns = bpf_ktime_get_ns();
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 },
};
static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;
```

```
    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }
    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        goto cleanup;
    }

    printf("evo_policy (FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

pc320 (jack-sparrow): best policy #68 6b62bff1 — score 607.1, 607.1 ops/s

Agent's title: region-freq CLOCK pass thresholds (1,3,8) -> (2,4,16) on team-best nemo 51067b5b (593.6 on pc320). Untouched knob: first pass now also evicts regions read twice (more FIFO-like for lukewarm regions, LevelDB block cache flattens page-level freq)

Closest seed by line similarity: fifo 77%, s3fifo 52%, mru 46%. Lines that differ from fifo are highlighted.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// [jack-sparrow: nemo 51067b5b with CLOCK pass thresholds (1,3,8) -> (2,4,16)]
// [captain-nemo: davy-jones f578a1ba with GRACE_NS 1s -> 200ms, REGION_SHIFT 4 -> 3]
// History-LFU (davy-jones): one list, CLOCK-style threshold passes (iterate API) evicting
// low-frequency folios.
// Frequency lives in a fixed-size counter array keyed by hash(inode, page index), so it
// SURVIVES eviction. Accesses to the same page within 2ms count once (the scan thread
// reads many keys per page in a burst, which otherwise pins scan pages): a hot page that is evicted and re-read keeps
// per-folio map. No per-folio metadata map at all. Counters are halved every AGE_PERIOD
// accesses (lazy, via an epoch stamp per slot) so the history slowly forgets.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define SKETCH_BITS 19
#define REGION_SHIFT 3      /* 8 pages = 32KB regions */
#define SKETCH_SIZE (1 << SKETCH_BITS)
#define SKETCH_MASK (SKETCH_SIZE - 1)
#define AGE_SHIFT 18        /* epoch advances every 2^18 accesses */
#define SAMPLE_SIZE 16
#define FREQ_MAX 0xffff
#define SCORE_MAX (0x7fffffffffffffffLL)

#define DEDUPE_NS (2ULL * 1000 * 1000)  /* bursts within 2ms count once */

struct dj_freq_slot {
    u32 count;
    u32 epoch;
    u64 last_ns;
    u64 added_ns;
};

#define GRACE_NS (200ULL * 1000 * 1000) /* new folios are not victims for 200ms */

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __type(key, u32);
    __type(value, struct dj_freq_slot);
    __uint(max_entries, SKETCH_SIZE);
} sketch SEC(".maps");

static u64 main_list;
static u64 nr_accesses = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline u32 folio_slot(struct folio *folio)
{
    u64 ino = folio->mapping->host->i_ino;
    // Region granularity: all pages of a 2^REGION_SHIFT-page file region share one counter,
    // so a readahead batch is kept or evicted as a unit and refaults come back as one IO.
    u64 region = (u64)folio->index >> REGION_SHIFT;
    u64 h = (ino * 0x9E3779B97F4A7C15ULL) ^ (region * 0xC2B2AE3D27D4EB4FULL);
    h ^= h >> 29;
    return (u32)(h & SKETCH_MASK);
}

static inline struct dj_freq_slot *get_slot(struct folio *folio, u32 epoch)
{
    u32 key = folio_slot(folio);
    struct dj_freq_slot *s = bpf_map_lookup_elem(&sketch, &key);
    if (!s)
        return NULL;
    u32 old = s->epoch;
    if (old != epoch) {
        u32 d = epoch - old;
        s->count = d >= 16 ? 0 : (s->count >> d);
        s->epoch = epoch;
    }
    return s;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    return 0;
}

static inline s64 folio_count(struct folio *f)
{
    if (!f->mapping || !f->mapping->host)
        return 0;
    struct dj_freq_slot *s = get_slot(f, (u32)(nr_accesses >> AGE_SHIFT));
    if (!s)
        return 0;
    if (bpf_ktime_get_ns() - s->added_ns < GRACE_NS)
        return 1LL << 40;
    return s->count;
}

// CLOCK-style pass over the list: evict folios whose (persistent) count <= thr, move the
// rest to the tail. Uses list_iterate instead of list_sample (every sample-based policy
// on the team scores 500-550, every iterate-based one >= 600).
#define LFU_ITER_FN(thr)                                \
static int lfu_iter_fn_##thr(int idx, struct cache_ext_list_node *a)            \
{                                                       \
    struct folio *f = a->folio;                         \
    if (!folio_test_uptodate(f) || !folio_test_lru(f))              \
        return CACHE_EXT_CONTINUE_ITER;                 \
    if (folio_test_dirty(f) || folio_test_writeback(f))             \
        return CACHE_EXT_CONTINUE_ITER;                 \
    if (folio_count(f) <= thr)                          \
        return CACHE_EXT_EVICT_NODE;                    \
    return CACHE_EXT_CONTINUE_ITER;                     \
}

LFU_ITER_FN(2)
LFU_ITER_FN(4)
LFU_ITER_FN(16)

static int any_iter_fn(int idx, struct cache_ext_list_node *a)
{
    struct folio *f = a->folio;
    if (!folio_test_uptodate(f) || !folio_test_lru(f))
        return CACHE_EXT_CONTINUE_ITER;
    if (folio_test_dirty(f) || folio_test_writeback(f))
        return CACHE_EXT_CONTINUE_ITER;
    return CACHE_EXT_EVICT_NODE;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_2, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_4, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
```

```
    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, lfu_iter_fn_16, &opts, eviction_ctx) < 0)
        return;
    if (eviction_ctx->nr_folios_to_evict >= eviction_ctx->request_nr_folios_to_evict)
        return;
    bpf_cache_ext_list_iterate_extended(memcg, main_list, any_iter_fn, &opts, eviction_ctx);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio)
{
    if (!is_folio_relevant(folio))
        return;
    u64 n = __sync_fetch_and_add(&nr_accesses, 1);
    struct dj_freq_slot *s = get_slot(folio, (u32)(n >> AGE_SHIFT));
    if (!s)
        return;
    u64 now = bpf_ktime_get_ns();
    if (now - s->last_ns < DEDUPE_NS)
        return;
    s->last_ns = now;
    if (s->count < FREQ_MAX)
        s->count += 1;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
}

void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    if (bpf_cache_ext_list_add_tail(main_list, folio)) {
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }
    struct dj_freq_slot *s = get_slot(folio, (u32)(nr_accesses >> AGE_SHIFT));
    if (s)
        s->added_ns = bpf_ktime_get_ns();
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
    char *cgroup_path;
    uint64_t cgroup_size;   /* accepted for harness compatibility */
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
    { "cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)" },
    { "cgroup_size", 's', "SIZE", 0,
      "Size of the cgroup in bytes (accepted for harness compatibility)" },
                    { 0 },
};

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
    case 'c':
        args->cgroup_path = arg;
        break;
    case 's':
        errno = 0;
        args->cgroup_size = strtoull(arg, NULL, 10);
        if (errno)
            args->cgroup_size = 0;
        break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }

    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
```

```
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        goto cleanup;
    }
    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        goto cleanup;
    }

    printf("evo_policy (FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```